Архитектура вычислительных систем 2025

Домашнее задание №2

Вариант 5
Имплементация модуля cnt_en_clr

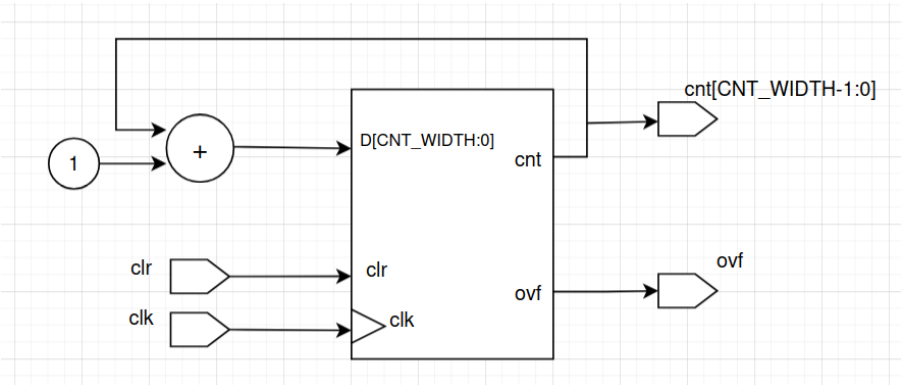
Шептяков Артём
Группа: Б01-207а
Почта: sheptiakov.ad@phystech.edu

1 Функциональное описание

Модуль **cnt_en_clr** представляет собой счётчик с возможностью асинхронного сброса и очистки, поддерживающий переменную ширину счётчика **CNT_WIDTH**. Основные функции модуля:

- Модуль управляется входным тактовым сигналом **clk_i** и работает по его положительному фронту.
- Асинхронный сброс выходных данных в 0 осуществляется с помощью сигнала **arstn_i** по его отрицательному фронту.
- Активация режима счета осуществляется сигналом **cnt_en_i**. Если сигнал активен в текущем такте и не выполняется очистка, то значение счётчика увеличивается на 1.
- Если значение счётчика достигает максимального значения $2^{CNT_WIDTH} - 1$, то при следующем такте счётчик сбрасывается в 0, а сигнал переполнения **ovf_o** устанавливается в 1 и может быть сброшен только асинхронным сбросом или сигналом очистки.
- Очистка выполняется сигналом **cnt_clr_i**. Если этот сигнал активен в текущий такт, то на следующий такт счётчик **cnt_o** и флаг переполнения **ovf_o** сбрасываются в 0.

2 Структурная схема



3 Параметры

Название	Описание	Допустимые значения
CNT_WIDTH	Разрядность счётчика	≥ 1

4 Порты

Название	Ширина	Описание
clk_i	1	Тактовый сигнал
arstn_i	1	Асинхронный сброс
cnt_en_i	1	Активация счета
cnt_clr_i	1	Очистка счётчика
cnt_o	CNT_WIDTH	Значение счётчика
ovf_o	1	Переполнение

5 Тактирование и сброс

Модуль управляется тактовым сигналом **clk_i** (по положительному фронту). Сброс происходит асинхронно по отрицательному фронту arstn_i. Сбрасываются в 0 значение счётчика **cnt_o** и флаг переполнения **ovf_o**.

6 Тестирование

Тестирование проводится с помощью testbench модуля, который включает следующий сценарий:

1. Инициализация сигналов: тактовый сигнал **clk_i** генерируется с периодом 2 нс, остальные сигналы устанавливаются в 0.

2. Проведение асинхронного сброса: **arstn_i** устанавливается в 0 и затем в 1, проверяется сброс счётчика.

3. Проверка счёта: **cnt_en_i** устанавливается в 1, и счётчик инкрементируется.

4. Остановка инкрементации: **cnt_en_i** сбрасывается в 0, значение счётчика остаётся неизменным.

5. Переполнение: счётчик доходит до максимального значения, после чего сбрасывается в 0, а **ovf_o** устанавливается в 1.

6. Очистка: **cnt_clr_i** устанавливается в 1, проверяется сброс счётчика и **ovf_o** в 0.

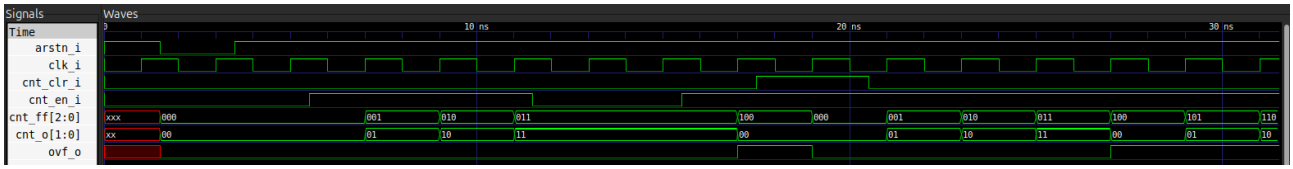
7. Периодический счет: после сброса счётчик начинает считать заново с 0.

7 Имплементация cnt_en_clr

```
module cnt_en_clr #(
    parameter CNT_WIDTH = 2
) (
    input clk_i,
    input arstn_i,
    input cnt_en_i,
    input cnt_clr_i,

    output [CNT_WIDTH-1:0] cnt_o,
    output ovf_o
);
    reg [CNT_WIDTH:0] cnt_ff;

    always @(posedge clk_i or negedge arstn_i) begin
        if (!arstn_i | cnt_clr_i) begin
            cnt_ff <= {CNT_WIDTH+1{1'b0}};
        end else begin
            cnt_ff <= cnt_en_i ? cnt_ff + 1 : cnt_ff;
        end
    end

    assign {ovf_o, cnt_o} = cnt_ff;
endmodule
```

8 Имплементация cnt_en_clr_tb

```
`timescale 1ns/1ps

module cnt_en_clr_tb();
    localparam CNT_WIDTH = 2;

    reg clk_i;
    reg arstn_i;
    reg cnt_en_i;
    reg cnt_clr_i;

    wire [CNT_WIDTH-1:0] cnt_o;
    wire ovf_o;

    cnt_en_clr #(
        .CNT_WIDTH(CNT_WIDTH)
    ) cnt_en_clr_inst (
        .clk_i    (clk_i    ),
        .arstn_i  (arstn_i  ),
        .cnt_en_i (cnt_en_i ),
        .cnt_clr_i(cnt_clr_i),

        .cnt_o (cnt_o),
        .ovf_o (ovf_o)
    );

    always begin
        clk_i = 1'b0; #1;
        clk_i = 1'b1; #1;
    end

    initial begin
        $dumpvars;
        arstn_i   = 1'b1;
        cnt_en_i  = 1'b0;
        cnt_clr_i = 1'b0;

        @(posedge clk_i); #0.5;
        arstn_i = 1'b0;

        @(posedge clk_i); #0.5;
        arstn_i = 1'b1;

        @(posedge clk_i); #0.5;
        cnt_en_i  = 1'b1; #2;
        @(posedge clk_i);
        @(posedge clk_i); #0.5;
        cnt_en_i  = 1'b0; #2;
        @(posedge clk_i); #0.5;
        cnt_en_i  = 1'b1;

        @(posedge clk_i); #0.5;
```

```
        cnt_clr_i  = 1'b1;
        @(posedge clk_i); #1.5;
        cnt_clr_i  = 1'b0; #10;
        @(posedge clk_i); #0.5;
        $finish;
    end
endmodule
```

9 Результаты testbench